

SonarQube Java Web Application Testing

Objective

The objective of this task is to set up a SonarQube environment to test and analyze a Java Web Application using two separate servers:

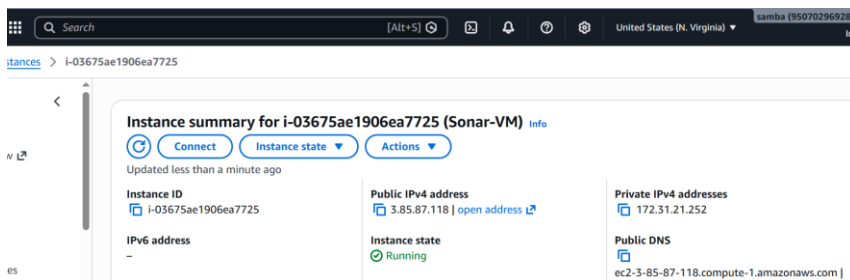
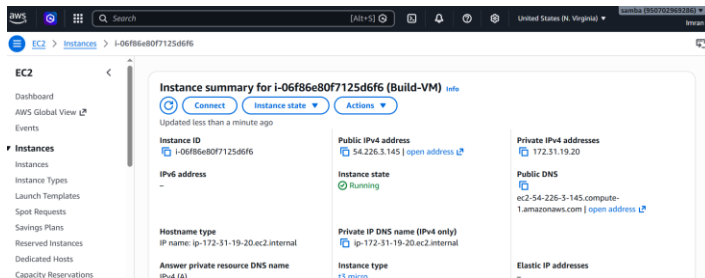
1. Test Server

- Used for building and testing the Java application.
- Contains Java, Maven, and the cloned application source code.

2. SonarQube Server

- Used to host SonarQube for code quality analysis.
- Runs SonarQube service on port 9000.

Step 1 – Create Two EC2 Servers



Step 2 – Configure Security Groups

Test Server

Allow:

- SSH Port 22

Security group name Sql-ssh	Security group ID sg-0c6ce06f706a1f83a	Description allow sql	VPC ID vpc-00b571da0ed5ea41d
Owner 950702969286	Inbound rules count 3 Permission entries	Outbound rules count 1 Permission entry	

Inbound rules

Outbound rules

Sharing

VPC associations

Related resources - new

Tags

Inbound rules (3)

Manage tags

Edit inbound rules

Q Search

Security group rule ID	IP version	Type	Protocol	Port range
sgr-0117584d6ad13a5af	IPv4	SSH	TCP	22

SonarQube Server

Allow:

- SSH Port 22
- Custom TCP Port 9000

This allows access to SonarQube from the browser.

Details			
Security group name ssh-sonar	Security group ID sg-09d8d5318b9ffee5f	Description allow ssh and sonar	VPC ID vpc-00b571da0ed5ea41d
Owner 950702969286	Inbound rules count 2 Permission entries	Outbound rules count 1 Permission entry	

Inbound rules

Outbound rules

Sharing

VPC associations

Related resources - new

Tags

Inbound rules (2)

Manage tags

Edit inbound rule

Q Search

Security group rule ID	IP version	Type	Protocol	Port range
sgr-070d24850f40bafa1	IPv4	Custom TCP	TCP	9000
sgr-0a9649e1901c31be0	IPv4	SSH	TCP	22

Step 3 – Install Java on Test Server

`sudo apt update`

Install Java:

`sudo apt install openjdk-17-jdk -y`

Verify Java:

`java -version`

```
ubuntu@Build-VM:~$ java --version
openjdk 17.0.18 2026-01-20
OpenJDK Runtime Environment (build 17.0.18+8-Ubuntu-1)
OpenJDK 64-Bit Server VM (build 17.0.18+8-Ubuntu-1, mixed mode, sharing)
ubuntu@Build-VM:~$ mvn --version
Apache Maven 3.9.12
Maven home: /usr/share/maven
Java version: 17.0.18, vendor: Ubuntu, runtime: /usr/lib/jvm/java-17-openjdk-amd64
Default locale: en, platform encoding: UTF-8
OS name: "linux", version: "7.0.0-1004-aws", arch: "amd64", family: "unix"
```

Step 4 – Install Maven on Test Server

Install Maven:

`sudo apt install maven -y`

Verify Maven:

`mvn -version`

```
ubuntu@Build-VM:~$ java --version
openjdk 17.0.18 2026-01-20
OpenJDK Runtime Environment (build 17.0.18+8-Ubuntu-1)
OpenJDK 64-Bit Server VM (build 17.0.18+8-Ubuntu-1, mixed mode, sharing)
ubuntu@Build-VM:~$ mvn --version
Apache Maven 3.9.12
Maven home: /usr/share/maven
Java version: 17.0.18, vendor: Ubuntu, runtime: /usr/lib/jvm/java-17-openjdk-amd64
Default locale: en, platform encoding: UTF-8
OS name: "linux", version: "7.0.0-1004-aws", arch: "amd64", family: "unix"
```

Step 5 – Clone Java Application from GitHub

Clone the repository:

git clone

```
ubuntu@Build-VM:~$ git clone https://github.com/mrtechreddy/Java-Web-Calculator-App.git
Cloning into 'Java-Web-Calculator-App'...
remote: Enumerating objects: 229, done.
remote: Counting objects: 100% (12/12), done.
remote: Compressing objects: 100% (10/10), done.
remote: Total 229 (delta 5), reused 2 (delta 2), pack-reused 217 (from 5)
Receiving objects: 100% (229/229), 66.03 KiB | 6.00 MiB/s, done.
Resolving deltas: 100% (53/53), done.
```

```
ubuntu@Build-VM:~$ ls
Java-Web-Calculator-App
ubuntu@Build-VM:~$ cd Java-Web-Calculator-App/
ubuntu@Build-VM:~/Java-Web-Calculator-App$ ls
Jenkinsfile  README.md  pom.xml  src
ubuntu@Build-VM:~/Java-Web-Calculator-App$ |
```

Step 6 – Install Java on SonarQube Server

```
ubuntu@Build-VM:~$ java --version
openjdk 17.0.18 2026-01-20
OpenJDK Runtime Environment (build 17.0.18+8-Ubuntu-1)
OpenJDK 64-Bit Server VM (build 17.0.18+8-Ubuntu-1, mixed mode, sharing)
ubuntu@Build-VM:~$ mvn --version
Apache Maven 3.9.12
Maven home: /usr/share/maven
Java version: 17.0.18, vendor: Ubuntu, runtime: /usr/lib/jvm/java-17-openjdk-amd64
Default locale: en, platform encoding: UTF-8
OS name: "linux", version: "7.0.0-1004-aws", arch: "amd64", family: "unix"
```

Step 7 – Download and Install SonarQube

Download SonarQube:

wget <https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-9.9.1.69595.zip>

```
ubuntu@Sonar-VM:~$ sudo wget https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-10.6.0.92116.zip
--2026-05-20 04:54:18-- https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-10.6.0.92116.zip
Resolving binaries.sonarsource.com (binaries.sonarsource.com)... 3.167.37.95, 3.167.37.88, 3.167.37.10, ...
Connecting to binaries.sonarsource.com (binaries.sonarsource.com)|3.167.37.95|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 731034761 (697M) [binary/octet-stream]
Saving to: 'sonarqube-10.6.0.92116.zip'

sonarqube-10.6.0.92116.zip  100%[=====] 697.17M  435MB/s  in 1.6s
2026-05-20 04:54:19 (435 MB/s) - 'sonarqube-10.6.0.92116.zip' saved [731034761/731034761]
```

Install unzip:

sudo apt install unzip -y

```
ubuntu@Sonar-VM:~$ sudo unzip sonarqube-10.6.0.92116.zip
Archive:  sonarqube-10.6.0.92116.zip
  creating: sonarqube-10.6.0.92116/
  creating: sonarqube-10.6.0.92116/jres/
  inflating: sonarqube-10.6.0.92116/jres/OpenJDK17U-jre_x64_linux_hotspot_17.0.11_9.tar.gz
  inflating: sonarqube-10.6.0.92116/jres/OpenJDK17U-jre_aarch64_linux_hotspot_17.0.11_9.tar.gz
  inflating: sonarqube-10.6.0.92116/jres/OpenJDK17U-jre_x64_alpine-linux_hotspot_17.0.11_9.tar.gz
```

Step 8 – Start SonarQube Service

Move to SonarQube bin directory:

cd ~/sonarqube/bin/linux-x86-64

Start SonarQube:

./sonar.sh start

Check status:

./sonar.sh status

If running successfully:

SonarQube is running

```
ubuntu@Sonar-VM:~$ cd sonarqube/
ubuntu@Sonar-VM:~/sonarqube$ ls
COPYING  bin  conf  data  dependency-license.json  elasticsearch  extensions  jres  lib  logs  temp  w
ubuntu@Sonar-VM:~/sonarqube$ cd bin/
ubuntu@Sonar-VM:~/sonarqube/bin$ ls
elasticsearch  linux-x86-64  macosx-universal-64  windows-x86-64  winsw-license
ubuntu@Sonar-VM:~/sonarqube/bin$ cd linux-x86-64/
ubuntu@Sonar-VM:~/sonarqube/bin/linux-x86-64$ ls
SonarQube.pid  sonar.sh
ubuntu@Sonar-VM:~/sonarqube/bin/linux-x86-64$ ./sonar.sh status
/usr/bin/java
rm: cannot remove './SonarQube.pid': Permission denied
Removed stale pid file: ./SonarQube.pid
SonarQube is not running.
ubuntu@Sonar-VM:~/sonarqube/bin/linux-x86-64$ ./sonar.sh start
/usr/bin/java
Starting SonarQube...
rm: cannot remove './SonarQube.pid': Permission denied
Removed stale pid file: ./SonarQube.pid
./sonar.sh: 165: cannot create ./SonarQube.pid: Permission denied
./sonar.sh: 164: cannot create ../../logs/nohup.log: Permission denied
rm: cannot remove './SonarQube.pid': Permission denied
Removed stale pid file: ./SonarQube.pid
Failed to start SonarQube.
ubuntu@Sonar-VM:~/sonarqube/bin/linux-x86-64$ sudo ./sonar.sh start
/usr/bin/java
Starting SonarQube...
Removed stale pid file: ./SonarQube.pid
Started SonarQube.
```

Step 9 – Access SonarQube Web UI

Open browser:

http://<SONARQUBE-SERVER-IP>:9000

Default Credentials:

Username: admin

Password: admin



Log in to SonarQube

Login *

Password *

[Go back](#)

[Log in](#)

Step 12 – Create SonarQube Token

1. Login to SonarQube
2. Click profile icon → My Account
3. Open Security tab
4. Under Tokens section:
 - Enter token name
 - Click Generate
5. Copy the generated token

Example:

squ_06abeefd6bfaf47b4b169a227ab50891c3e39e4eThis token is used by the Build/Test Server to authenticate with SonarQube.

Tokens of *Administrator*

Name	Type	Project	Last use	Created	Expiration	
sonar	User		< 1 hour ago	May 20, 2026	June 19, 2026	Revoke

Step 13 – Configure SonarQube Analysis on Build Server

Move to project directory:

```
cd <PROJECT-FOLDER>
```

Run SonarQube analysis:

```
mvn sonar:sonar \
-Dsonar.projectKey=java-web-app \
-Dsonar.host.url=http://<SONARQUBE-SERVER-IP>:9000 \
-Dsonar.login=<SONAR-TOKEN>
```

```
ubuntu@Build-VM:~/Java-Web-Calculator-App$ mvn clean verify sonar:sonar \
-Dsonar.projectKey=my-java-app \
-Dsonar.host.url=http://3.85.87.118:9000 \
-Dsonar.login=squ_06abee6d6bfaf47b4b169a227ab50891c3e39e4e
[INFO] Scanning for projects...
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-metadata.xml
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/mojo/maven-metadata.xml
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/mojo/maven-metadata.xml (21 kB at 34 kB/s)
```

Step 14 – Verify SonarQube Analysis

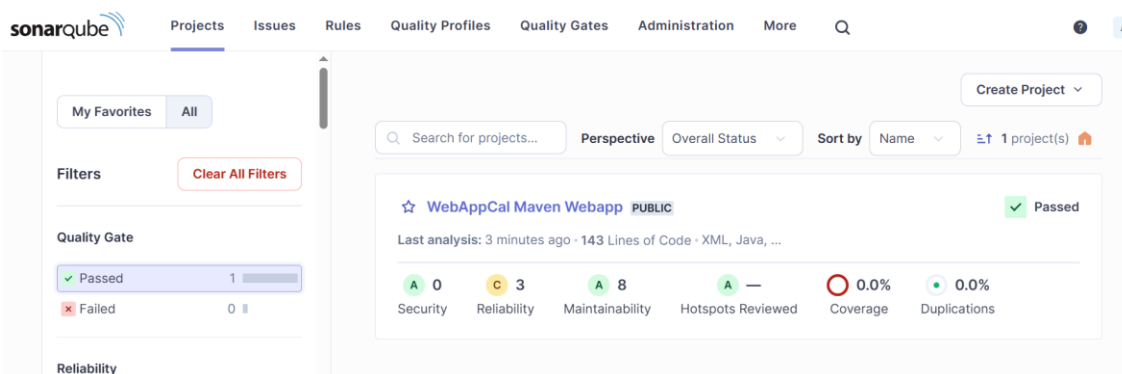
After successful execution:

BUILD SUCCESS

```
c374b5c9838
[INFO] 08:59:44.464 Analysis total time: 12.458 s
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 22.061 s
[INFO] Finished at: 2026-05-20T08:59:44Z
[INFO] -----
ubuntu@Build-VM:~/Java-Web-Calculator-App$ |
```

Open SonarQube dashboard:

<http://3.85.87.118/9000>



Conclusion

Successfully configured a two-server SonarQube environment for testing and analyzing a Java Web Application. The Build/Test Server was used to build the application using Java and Maven, while the SonarQube Server performed code quality analysis through port 9000. Authentication

between both servers was established using a generated SonarQube token. The setup helped identify code quality issues, vulnerabilities, and maintainability metrics effectively.